Avaliação A2 - Paradigmas de Linguagens de Programação

Grupo 05

👥 Integrantes (3 a 5 alunos)

- Gustavo da Costa Matsuo (33646155)
- Kauã Covo Vieira (33928053)
- Luiz Gustavo Caldeira Ribeiro (33051925)
- Maria Isabella Nashimoto de Andrade (34101012)
- Paulo Henrique Santos Souza (33790671)

Definição do Tema

- **Paradigma:** Lógico
- **Linguagem:** Prolog
- **Código do Tema:** L1
- **Descrição do Desafio:** Base de conhecimento genealógica

Execução do Código

- **Ambiente Online Utilizado:** SWISH
- **Permalink:** [https://swish.swi-prolog.org/p/ArvoreGenealogicaProlog.pl]

Instruções de Teste (Entrada e Saída)

Ao clicar no link e abrir a tela do ambiente online, haverá um espaço de digitação no canto inferior direito que possui ?-. Nesta área, escreva: " mostrar_relacoes(). ".

Dentro do parênteses, escreva o nome de uma pessoa que esteja na árvore para verificar quais relações a pessoa tem dentro da árvore.

Após isso, aparecerá logo abaixo a opção RUN, clique nele e o código retornará com quais pessoas o nome digitado tem relação e qual o tipo de parentesco.

Entrada de Exemplo:

```
None
mostrar_relacoes(nicole).
```

Saída Esperada:

```
None
Pessoa: nicole
Parentes:
  Pai: diego
  Mãe: sarah
  Esposo(a): [yuri]
  Filhos: [samira]
  Irmãos: [leonardo]
  Irmãs: Nenhum
  Avô: [davi]
  Avó: [antonella]
  Tios: Nenhum
  Tias: Nenhum
  Primos: Nenhum
```

Relatório Técnico

Descrição da Solução

O problema proposto foi resolvido por meio da modelagem e implementação de uma árvore genealógica utilizando duas abordagens distintas para fins comparativos: o paradigma lógico baseado em regras no Prolog e o paradigma orientado a objetos em Python.

1. Modelagem da Base de Conhecimento e Estrutura de Dados Em Prolog: A base de fatos foi estruturada utilizando o predicado binário progenitor(PaiOuMae, Filho) para registrar de forma unívoca a ascendência entre os indivíduos, complementada pelo fato estático sexo(Pessoa, Genero) para classificar rigidamente cada indivíduo como masculino ou feminino.

Em Python: A arquitetura encapsulou os dados dentro de uma classe dedicada (ArvoreGenealogica), armazenando as relações de parentesco em conjuntos estruturados (set de tuplas (progenitor, filho)) e os atributos demográficos em dicionários de mapeamento rápido (dict).

2. Arquitetura de Regras Lógicas e Relações Derivadas A partir dos fatos básicos, construiu-se uma hierarquia modular de regras lógicas:

Parentescos Primários e Diretos: Definição de regras para pai/2, mae/2 e filho_de/2 por meio da inversão e filtragem de gênero sobre os fatos de progenitor.

Irmandade e Correção de Duplicatas por Dupla Ascendência: Como cada indivíduo possui nativamente dois progenitores (pai e mãe), consultas diretas geram duplicações por backtracking. Para solucionar isso, desenvolveu-se o predicado intermediário irmao_base/2, que valida o compartilhamento de progenitores, refinando-se depois em irmao/2 e irma/2. O uso da biblioteca nativa setof/3 garantiu o agrupamento e eliminação de duplicatas.

Ascendência Estendida e Colaterais: Implementação de regras para avós (vô/2, vó/2), tios consanguíneos e tios por afinidade (incorporando a regra de conjuge/2 baseada em filhos em comum) e primos (primo/2).

3. Automatização de Relatórios Textuais Verticais Para atender ao requisito de exibir todas as relações de um indivíduo de uma só vez, implementou-se o predicado unificado mostrar_relacoes/1 em Prolog (análogo à Opção 2 do menu interativo em Python). Este algoritmo varre de forma limpa todas as categorias de parentesco e imprime um relatório estruturado no terminal.

Análise Comparativa

1. Sintaxe

O Prolog adota um paradigma estritamente declarativo, onde o desenvolvedor declara fatos e regras de relação (ex: filho_de(X, Y) :- progenitor(Y, X).) e o motor de inferência resolve as consultas. O Python adota o paradigma imperativo e orientado a objetos, exigindo o controle procedural explícito do fluxo de execução. Enquanto a verificação de parentesco em Prolog ocorre por unificação direta de variáveis nas regras lógicas, em Python é necessário escrever compreensões de listas ou varreduras condicionais explícitas. **Olhar Imagens 1 e 2 do repositório.**

2. Semântica

Atribuição vs. Unificação: Em linguagens imperativas e orientadas a objetos como o Python, a semântica de atribuição (=) opera como a vinculação de uma referência de memória a um objeto mutável ou imutável em um escopo local ou global explícito. Já o Prolog opera sob o princípio da unificação lógica baseada no casamento de padrões (pattern matching). No Prolog, as variáveis (iniciadas por letras maiúsculas, como X ou Y) não guardam valores atribuídos sequencialmente; elas representam termos lógicos que tentam encontrar uma correspondência estrutural com os fatos e regras da base de conhecimento para tornar a proposição verdadeira.

Forma de Avaliação (Retrocesso vs. Execução Sequencial): O modelo de avaliação do Python é estritamente estipulado por um fluxo procedural e determinístico, onde cada instrução

é executada linha por linha na ordem ditada pelo programador. Em contrapartida, o Prolog utiliza um motor de inferência baseado em busca em profundidade com retrocesso automático (backtracking). Quando uma regra falha, o interpretador retorna automaticamente ao ponto de escolha anterior para testar caminhos alternativos de unificação. Esse mecanismo exigiu cuidados específicos no projeto, como o uso de predicados de agregação (setof/3), para lidar com caminhos múltiplos na árvore genealógica (por exemplo, a dupla ascendência paterna e materna) que geravam respostas redundantes no console.

Bidirecionalidade e Escopo de Relações: Semanticamente, as regras declarativas do Prolog possuem bidirecionalidade nativa. Uma única regra definida como progenitor(X, Y) serve de forma universal tanto para consultar quem são os filhos de um determinado pai ou mãe (passando X fixo e buscando Y) quanto para descobrir quem são os progenitores de um indivíduo (passando Y fixo e buscando X). No Python, o escopo semântico dos métodos de classes é unidirecional, exigindo a implementação procedural de lógicas distintas, filtragens por compreensões de listas ou varreduras direcionadas em dicionários e conjuntos para atender a cada sentido específico de busca.

3. Gerenciamento de Memória

O Prolog gerencia a memória internamente através da sua base de dados dinâmica de fatos e pilhas de execução de pilhas/trilhas (trail and local stacks) necessárias para suportar o mecanismo de backtracking.

O Python (CPython) gerencia memória por meio de alocação dinâmica em heap combinada com um coletor de lixo (garbage collector) baseado primordialmente em contagem de referências e detecção de ciclos.

Comparação com o C: Ao contrário da linguagem C — onde o programador é responsável pelo gerenciamento manual de ponteiros e chamadas explícitas a funções como malloc() e free() —, tanto o Prolog quanto o Python abstraem totalmente essa complexidade de baixo nível, eliminando riscos diretos de vazamentos de memória (memory leaks).

4. Trade-offs

Vantagens do Prolog: Extrema concisão e poder expressivo para modelar domínios complexos baseados em relações e restrições lógicas, desonerando o programador de escrever algoritmos de busca manuais.

Desvantagens do Prolog: Curva de aprendizado íngreme, controle de fluxo opaco (o comportamento de backtracking pode gerar repetições indesejadas que exigem tratamento com setof/3), e menor flexibilidade para construir interfaces de usuário ou menus interativos complexos.

Vantagens do Python: Excelente legibilidade, facilidade robusta para gerenciar entradas e saídas via terminal (menus interativos), tratamento avançado de strings (.lower().strip()) e facilidade de integração com ecossistemas de software modernos.

Desvantagens do Python: Exige um volume consideravelmente maior de código imperativo e lógica procedural para resolver problemas de inferência que no Prolog são deduzidos de forma nativa e declarativa.

Resumo da Análise Comparativa

- **Sintaxe:** O Prolog é declarativo baseando-se no que se deseja resolver por meio de fatos e regras lógicas, delegando a busca ao motor de inferência. O Python adota um estilo imperativo e orientado a objetos, exigindo algoritmos procedurais explícitos para percorrer estruturas.

- **Semântica:**

A semântica de uma linguagem define o significado e o comportamento dos comandos e estruturas em tempo de execução, revelando distinções marcantes entre o paradigma lógico do Prolog e o imperativo/orientado a objetos do Python.

Primeiramente, a manipulação de dados opera por lógicas opostas: enquanto o Python utiliza a atribuição explícita de referências a objetos em escopos locais ou globais, o Prolog fundamenta-se na unificação lógica e no casamento de padrões, onde variáveis em letras maiúsculas representam termos abstratos que buscam correspondência estrutural na base de conhecimento.

Quanto ao modelo de avaliação, o Python adota uma execução estritamente estrita e sequencial, ditada pelo fluxo procedural instrução por instrução. Em contraste, o Prolog utiliza um motor de inferência baseado em retrocesso automático (backtracking), retornando a pontos de escolha anteriores quando uma proposição falha — o que exigiu o uso de predicados de agregação para evitar redundâncias na árvore.

Por fim, destaca-se a bidirecionalidade: as regras em Prolog possuem bidirecionalidade nativa, servindo tanto para buscar ascendentes quanto descendentes com o mesmo predicado, ao passo que os métodos em Python são unidirecionais, exigindo lógicas procedurais dedicadas e filtragens programadas para cada direção de busca.

- **Gerenciamento de Memória:**

No Prolog: o gerenciamento é automático e feito pela máquina abstrata da linguagem (como a WAM — Warren Abstract Machine), alocando pilhas temporárias para controlar os estados de backtracking e a base de fatos dinâmicos. No Python: é utilizada a coleta de lixo (Garbage

Collection) baseada primordialmente em contagem de referências suplementada por detecção de ciclos para objetos no heap. Em comparação com o C que exige liberação manual com free e alocação com malloc, tanto o Prolog quanto o Python abstraem o gerenciamento de baixo nível para o programador.

- **Trade-offs:** (Vantagens e desvantagens do novo paradigma em relação aos anteriores)

Log de Uso de Inteligência Artificial (IA)

- **O que foi pedido à IA:**

Inicialmente, solicitou-se à inteligência artificial uma análise detalhada sobre o código inicial em Prolog da árvore genealógica para identificar e corrigir eventuais falhas estruturais, pontos soltos ou irregularidades na construção dos fatos. Posteriormente, o grupo pediu auxílio na formulação e no aprimoramento de algoritmos lógicos para mapear e extrair todas as relações de parentesco possíveis de um indivíduo — como pais, mães, irmãos, avós, tios e primos —, além da criação de rotinas específicas para exibir de forma unificada e vertical todas essas conexões de uma só vez. Um ponto central demandado foi a resolução do problema de duplicação de resultados gerados pelo motor de inferência devido à dupla ascendência (pai e mãe). Por fim, pediu-se a transposição de toda essa lógica relacional para um código equivalente em Python, estruturado com classes, coleções eficientes (set e dict) e menus interativos de terminal.

- **Qual IA foi utilizada:**

Google Gemini

- **O que foi aproveitado:**

Do suporte da IA, aproveitou-se integralmente a estruturação lógica das regras de inferência em Prolog — incluindo a criação de predicados intermediários de base (irmao_base/2) e o uso de predicados de agregação como setof/3 para suprimir duplicatas indesejadas decorrentes do backtracking. Também foram aproveitados os códigos funcionais adaptados, como o predicado de exibição vertical (mostrar_relacoes/1) e a arquitetura orientada a objetos em Python com o menu interativo formatado para listar de maneira limpa todos os parentescos de um indivíduo.

- **O que foi reescrito/entendido pelo grupo:**

O grupo absorveu conceitos fundamentais de programação lógica e paradigmas de computação que representavam um contato inédito com a linguagem Prolog. Por meio das explicações detalhadas, correções de sintaxe, exemplos de código e materiais de apoio fornecidos pela IA, os integrantes compreenderam a distinção entre a abordagem declarativa (baseada em fatos e regras unificadas) e a abordagem imperativa/orientada a objetos. Cada

trecho de código sugerido foi rigorosamente testado, validado e executado no ambiente online do SWI-Prolog e interpretadores Python, garantindo que o grupo dominasse a lógica por trás da modelagem genealógica e do tratamento de restrições de busca.